Architecture and Security Overview

ImplicitEx — Technical Review Packet

Document	v1.1.2 — 2026-07-24
Prepared by	Antoine Dennison, ImplicitEx — connect@implicitex.com
Reference	Blockaid Classification Review
Scope	Deployed production application — Polygon mainnet, chain ID 137

CONTENTS

1 EXECUTIVE SUMMARY

ImplicitEx is a non-custodial USDC transfer interface operating on Polygon (chain ID 137). It allows a sender to transfer USDC to a recipient address they specify. A platform fee of 1% is charged on each transfer. The production portal currently limits transfers to 250.00 USDC, making the highest fee reachable through the supported interface 2.50 USDC. The application does not hold, route through, or control user funds beyond what the user explicitly authorizes through their own wallet.

This document was prepared in response to a Blockaid classification concerning the ImplicitEx USDC approval request as a deceptive request. The sections below document the complete transaction lifecycle, deployed security controls, fee mechanics, and independently verifiable on-chain evidence available for review.

The July 23 transaction is the original Blockaid-observed event and is supported by contemporaneous screenshots and its public on-chain record. The July 24 approval-path and transfer-only transactions are separate controlled certification runs whose original browser-exported proof packets were preserved byte-for-byte at time of export.

Why the wallet requests an approval

USDC is an ERC-20 token. ERC-20 tokens cannot be spent by a smart contract without the token holder first granting a spending allowance to that contract. The `approve()` call is the standard ERC-20 mechanism for establishing this permission. The allowance requested equals exactly the amount the user has already reviewed on-screen: the recipient amount plus the platform fee. No additional buffer is requested.

Why the approval amount is exact rather than unlimited

Many applications request an unlimited or large blanket allowance to avoid requiring a new approval on each transaction. ImplicitEx does not do this. The allowance request is calculated per transaction and covers only the current transfer total. After the contract calls `transferFrom()`, the allowance returns to zero. The user's residual exposure is zero after each transaction completes.

Why the contract is non-custodial

The smart contract does not hold funds at any point in the transfer. The `transferWithFee()` function issues two `safeTransferFrom()` calls in a single atomic execution: one from the sender directly to the recipient, and one from the sender directly to the treasury. The contract is not an intermediate holder. If either call fails, the entire transaction reverts and no funds move.

> **AUDIT STATUS**
>
> This document does not constitute a third-party security assessment. ImplicitEx has not undergone an independent audit at this stage. The verification evidence presented in Section 8 is internal. This is stated explicitly in Section 9.

2 SYSTEM ARCHITECTURE

2.1 Component map

2.2 Domains

Domain	Role	Status
`https://implicitex.com`	Primary production domain	Live
`https://portal.implicitex.com`	Transfer portal, direct entry point	Live
`https://implicitex-236f2.web.app`	Firebase staging URL — included in original Blockaid report	Staging

2.3 Network parameters

Parameter	Value
Network	Polygon PoS Mainnet
Chain ID	137 (hex: `0x89`)
RPC endpoint	`polygon-bor-rpc.publicnode.com` (public, no key required)
Block explorer	`polygonscan.com`

3 TRANSACTION LIFECYCLE

The following describes a 1.00 USDC transfer. The same sequence applies at all amounts within the configured limits.

3.1 User input

The user enters a recipient Ethereum address and a transfer amount. No wallet connection is required at this stage.

3.2 Pre-flight validation (no wallet prompt)

The portal calls `previewTransfer(sender, amount)` on the contract. This is a read-only view call. It returns the calculated fee, total debit, sender USDC balance, current allowance, and a boolean indicating whether the transfer can proceed.

The portal also validates locally:

- Connected network matches Polygon mainnet (chain ID 137)
- Recipient address is non-zero and is an externally owned account (EOA), not a contract
- Amount falls within the configured transfer limits (1.00 USDC floor, 250.00 USDC ceiling)

No wallet action is initiated during this phase.

3.3 Review screen

Before any wallet prompt is triggered, the portal presents a summary of the pending transaction:

```
Recipient:      0xe0B02A6d...796B
Amount:         1.000000 USDC
Platform fee:   0.010000 USDC
Total debit:    1.010000 USDC
Network:        Polygon
```

The user must explicitly confirm past this screen to proceed. No wallet prompt has been issued at this point.

3.4 USDC spending approval (wallet prompt 1 of 2)

The portal calls `approve(contractAddress, totalDebit)` on the Circle USDC token contract (`0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359`).

MetaMask surfaces this as a "Set a spending cap for your USDC" dialog. The spending cap displayed equals the exact total debit shown on the review screen — 1.010000 USDC in this example. During the observed 2026-07-23 transaction documented in Section 7, MetaMask displayed a Blockaid warning at this stage.

The approval amount is not unlimited. It is not padded with a buffer. It matches the review screen figure precisely.

3.5 Transfer execution (wallet prompt 2 of 2)

After the approval transaction is confirmed on-chain, the portal calls `transferWithFee(recipientAddress, 1000000)` on the ImplicitExTransfer contract. MetaMask surfaces this as a separate contract interaction confirmation. The user must approve this second step independently.

3.6 On-chain execution

The contract executes atomically:

```
USDC.safeTransferFrom(sender, recipient, 1000000)   // 1.000000 USDC → recipient
USDC.safeTransferFrom(sender, treasury, 10000)      // 0.010000 USDC → treasury fee
```

If either call reverts, the entire transaction reverts. The contract retains nothing.

3.7 Confirmation

The portal polls for block confirmation. It displays a pending state until the transaction is confirmed on-chain, then transitions to a confirmed state. It does not report success at submission time.

3.8 Economic summary — 1.00 USDC example

Flow	Amount	Verified
Sender USDC deducted	1.010000 USDC	Gate 4 smoke (2026-06-15)
Recipient USDC received	1.000000 USDC	Gate 4 smoke, Polygonscan confirmed
Treasury fee received	0.010000 USDC	Gate 4 smoke, Polygonscan confirmed

Flow	Amount	Verified
Contract retained	0.000000 USDC	Gate 4 smoke, zero-custody confirmed

4 TRUST BOUNDARIES

4.1 What the application can and cannot do

Capability	Permitted	Notes
Read wallet address	Yes	Required to populate sender field
Request USDC allowance	Yes — exact amount only	ERC-20 standard mechanism; amount matches review screen
Execute `transferWithFee()`	Yes — requires separate user signature	Wallet presents a second, distinct confirmation
Access funds beyond approved amount	No	Constrained by ERC-20 allowance model
Bypass wallet confirmation	No	Every on-chain action requires a valid user signature
Move funds without user signature	No	No server-side signing; no key storage
Hold user funds in contract	No	Direct sender→recipient and sender→treasury routing
Withdraw accumulated contract balance	No	Contract holds no USDC balance; no withdrawal function
Request unlimited USDC allowance	No	Approval amount is computed per transaction
Reverse or refund a confirmed transaction	No	Blockchain transactions are final; no reversal mechanism exists

4.2 User authorization sequence

Every on-chain action requires a private-key signature from the user. The wallet mediates this. The portal cannot initiate, sign, or bypass any wallet action.

```
Portal      →  constructs transaction parameters
Wallet      →  displays parameters to user
User        →  signs or rejects
Blockchain  →  executes only if user signature is valid and allowance is sufficient
```

The portal has no mechanism to sign transactions on the user's behalf, store private keys, or initiate any transaction without explicit user action at the wallet prompt.

5 SMART CONTRACT BEHAVIOR

Contract name	`ImplicitExTransfer`
Solidity version	`^0.8.24`
License	MIT
Deployed address	`0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0`
Source file	`app-web/contracts/implicitex_transfer.sol`
Polygonscan source	Verified — `https://polygonscan.com/address/0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0#code`

5.1 Core function: `transferWithFee(address recipient, uint256 amount)`

This is the only function that moves funds. Execution sequence:

1. Rejects call if contract is paused (`whenNotPaused`)
2. Applies reentrancy guard (`nonReentrant`)
3. Rejects zero-address recipient
4. Rejects contract addresses as recipients (recipient must be EOA)
5. Enforces minimum transfer amount
6. Enforces transfer precision (amount must be divisible by configured precision)
7. Calculates fee: `(amount × feeBasisPoints) / 10000`
8. Calls `safeTransferFrom(sender, recipient, amount)`
9. Calls `safeTransferFrom(sender, treasury, fee)`
10. Emits `TransferExecuted` event

5.2 View function: `previewTransfer(address sender, uint256 amount)`

Read-only. Returns five values without any state change or gas cost beyond the RPC call:

Return value	Description
`fee`	Calculated platform fee for this amount
`totalDebit`	Amount + fee (the figure shown on the review screen)
`balance`	Sender's current USDC balance

Return value	Description
`allowance`	Current USDC allowance granted to the contract
`canTransfer`	Boolean — true if balance and allowance are sufficient

The portal calls this function before presenting the review screen and again immediately before initiating the approval step, to confirm on-chain state has not changed between input and execution.

5.3 Fee constants (compile-time constants; not administratively configurable)

Constant	Value	Description
`MAX_FEE_BPS`	100	Maximum fee rate, in basis points. 100 bps = 1.00%. The owner cannot set `feeBasisPoints` above this value. This is the only compile-time fee constraint in the deployed contract.

The contract limits the fee rate to a maximum of 100 basis points but does not enforce an absolute USDC transfer or fee-value ceiling. The fee formula is `(amount × feeBasisPoints) / 10000` with no cap on the resulting USDC amount. The supported production portal separately limits transfers to 250.00 USDC, making the maximum fee reachable through that interface 2.50 USDC. Direct contract callers are subject to the same deterministic percentage calculation but are not subject to the portal's amount ceiling. See Section 9 for disclosure of the future fee cap policy.

5.4 Events emitted

Event	Triggered by
`TransferExecuted(sender, recipient, amountSent, feeAmount, totalDebited)`	Every successful `transferWithFee()` call
`TreasuryUpdated(previousTreasury, newTreasury)`	Owner changes the fee recipient address
`FeeUpdated(previousFeeBps, newFeeBps)`	Owner changes the fee rate
`MinTransferUpdated(previous, new)`	Owner changes the minimum transfer floor
`PrecisionUpdated(previous, new)`	Owner changes the transfer precision setting
`TokensRescued(token, to, amount)`	Owner calls `rescueERC20` for a non-USDC token

5.5 Owner-controlled parameters

Function	Effect	Constraint
`setTreasury(address)`	Update the fee recipient address	Cannot be zero address

Function	Effect	Constraint
`setFeeBasisPoints(uint16)`	Adjust the fee rate	Cannot exceed `MAX_FEE_BPS` (100)
`setMinTransferAmount(uint256)`	Change the minimum transfer floor	Owner discretion
`setTransferPrecision(uint256)`	Change required precision divisor	Owner discretion
`pause()`	Halt all `transferWithFee()` calls	Owner-only
`unpause()`	Resume transfers	Owner-only
`rescueERC20(token, to, amount)`	Recover tokens sent to the contract by mistake	Explicitly reverts if `token == USDC` — USDC cannot be drained via this function

5.6 Ownership model

The contract uses OpenZeppelin `Ownable2Step`. An ownership transfer requires two transactions: the current owner nominates a new address, and that address must explicitly accept. This prevents accidental or unilateral ownership loss.

Current owner: `0x776A0D6b9F96445A38303F56d5B923e6d1FF8E97`

6 DEPLOYED SECURITY CONTROLS

The following controls are active in the currently deployed contract and portal. "Deployed" means the control is live in production and was verified during the Gate 4 controlled smoke (2026-06-15). "Portal-enforced" means the control is implemented in the frontend JavaScript and does not rely on the contract.

Control	Mechanism	Layer
Exact-amount allowance	`approve(contract, amount + fee)` — no unlimited approval, no buffer	Portal-enforced
Non-custodial routing	`safeTransferFrom(sender, recipient)` and `safeTransferFrom(sender, treasury)` — contract is never an intermediate holder	Contract
Reentrancy protection	`nonReentrant` modifier on `transferWithFee()`	Contract
Pause mechanism	`whenNotPaused` modifier; owner can halt all transfers	Contract
Zero-address rejection	Recipient validated as non-zero before `safeTransferFrom()`	Contract
EOA-only recipients	Contract addresses rejected as recipients	Contract

Control	Mechanism	Layer
USDC drain prevention	`rescueERC20` explicitly reverts when `token == USDC`	Contract
Transfer floor	`minTransferAmount` — 1.000000 USDC at current deployment	Contract
Transfer ceiling (soft launch)	250.000000 USDC per transaction — limits maximum reachable fee to 2.50 USDC at current 1% rate	Portal-enforced
Two-step ownership	`Ownable2Step` — ownership change requires explicit acceptance from new owner	Contract
Chain enforcement	Portal validates chain ID 137 before any wallet prompt	Portal-enforced
Global and per-chain transfer gates	As of 2026-07-22, the global `transfersEnabled` flag is `true`; Polygon mainnet (chain 137) is `true`; and Amoy testnet is `false`. These are frontend operating controls, not immutable contract limits. Contract-level `paused()` is separate and can be verified using the read-only query in Section 10.3.	Portal-enforced
Atomic execution	Both `safeTransferFrom` calls execute in one transaction; either both succeed or both revert	Contract
On-chain confirmation requirement	Portal displays pending state until block confirmation; does not report success at submission	Portal-enforced
Flow identity token	In-progress flow is invalidated on wallet account change or network change mid-session	Portal-enforced

7 OBSERVED TRANSACTION FLOW

This section documents the observed transaction flow and independently verifiable on-chain evidence for both supported execution paths. The 2026-07-23 transaction is the original Blockaid-observed event, preserved as a distinct historical record supported by contemporaneous screenshots and its public chain record. The 2026-07-24 approval-path and transfer-only transactions are separate controlled certification runs whose original browser-exported proof packets were preserved byte-for-byte at time of export and are not attributed to the earlier event.

OBSERVED TRANSACTION

Amount: 1.00 USDC · Platform fee: 0.01 USDC · Total debit: 1.01 USDC · Network: Polygon · Sender: `0xf614356f93408460b594addacc86a7fc94310f1d` · Recipient: `0x5466bbA8cD334554c88F81342dDfcEc4c4A7698B` · Receipt: IX-07FF5D03 · Block: 90,776,572

Stage	Observed evidence
1. Portal loaded	Production portal and Polygon chain data rendered successfully.
2. Wallet connection	MetaMask site connection request, sender address, and USDC balance.
3. Recipient validation	Recipient address accepted with "Format valid" status.
4. Fee calculation	1.00 USDC amount produced a 0.010000 USDC platform fee.
5. User confirmation	Execute Transfer remained disabled until the user checked the explicit acknowledgement.
6. ERC-20 approval request	MetaMask presented a 1.01 USDC spending-cap request.
7. Observed Blockaid warning	The approval prompt displayed "This is a deceptive request." The transfer proceeded after the user confirmed the approval.
8. Transfer transaction request	After approval, the portal issued the separate `transferWithFee()` call. The transfer was broadcast to Polygon.
9. On-chain confirmation	The transfer was confirmed at block 90,776,572. Funds moved.
10. Activity record	The portal recorded receipt IX-07FF5D03 as **CONFIRMED** with funds moved on Polygon.
11. Proof packet	The portal export recorded the transaction hash, block number, parties, amounts, and confirmed settlement fields.

7.1 Portal initialization and wallet connection

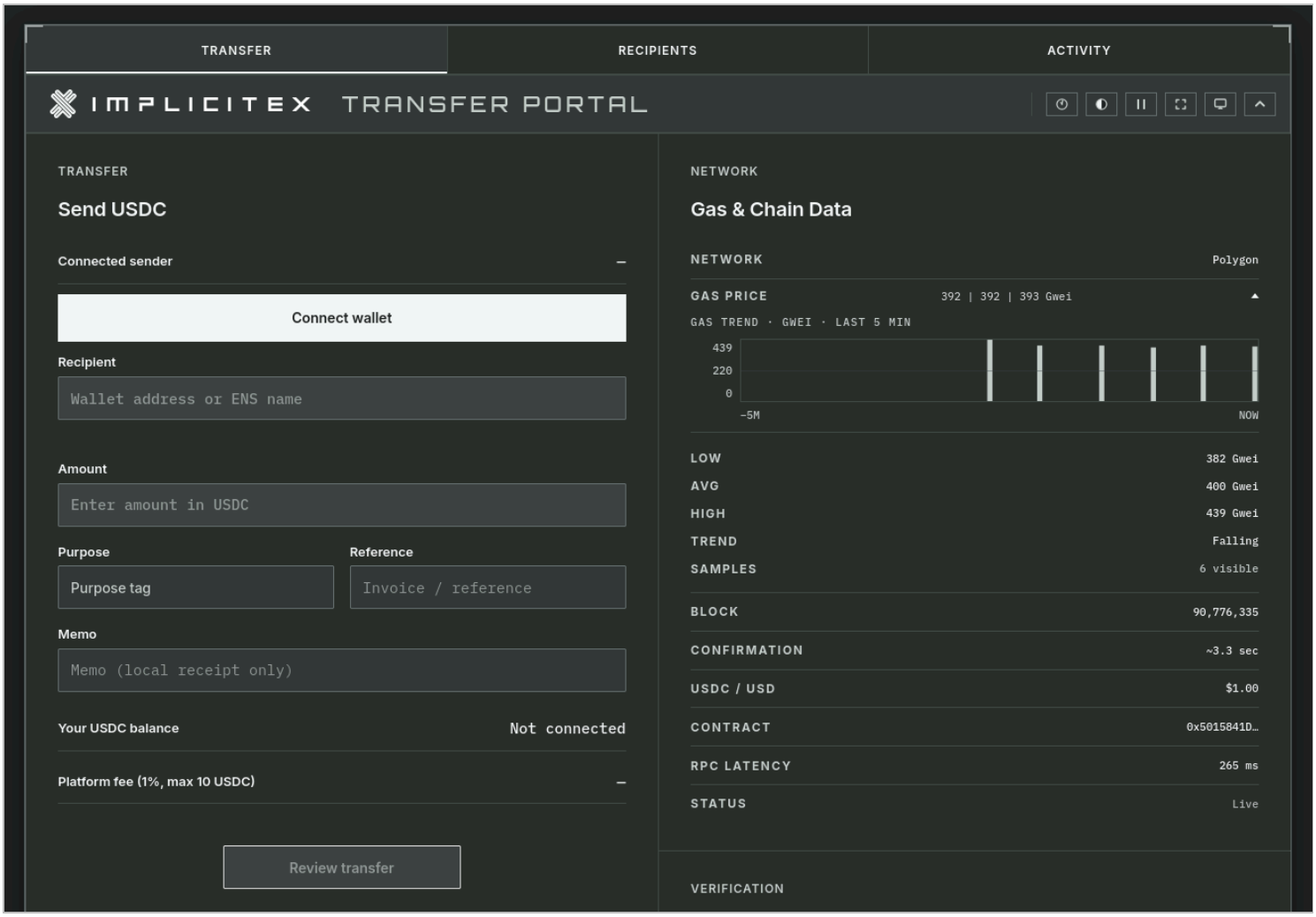

Figure 1

Portal initialization. The interface and live Polygon chain data rendered before wallet connection.

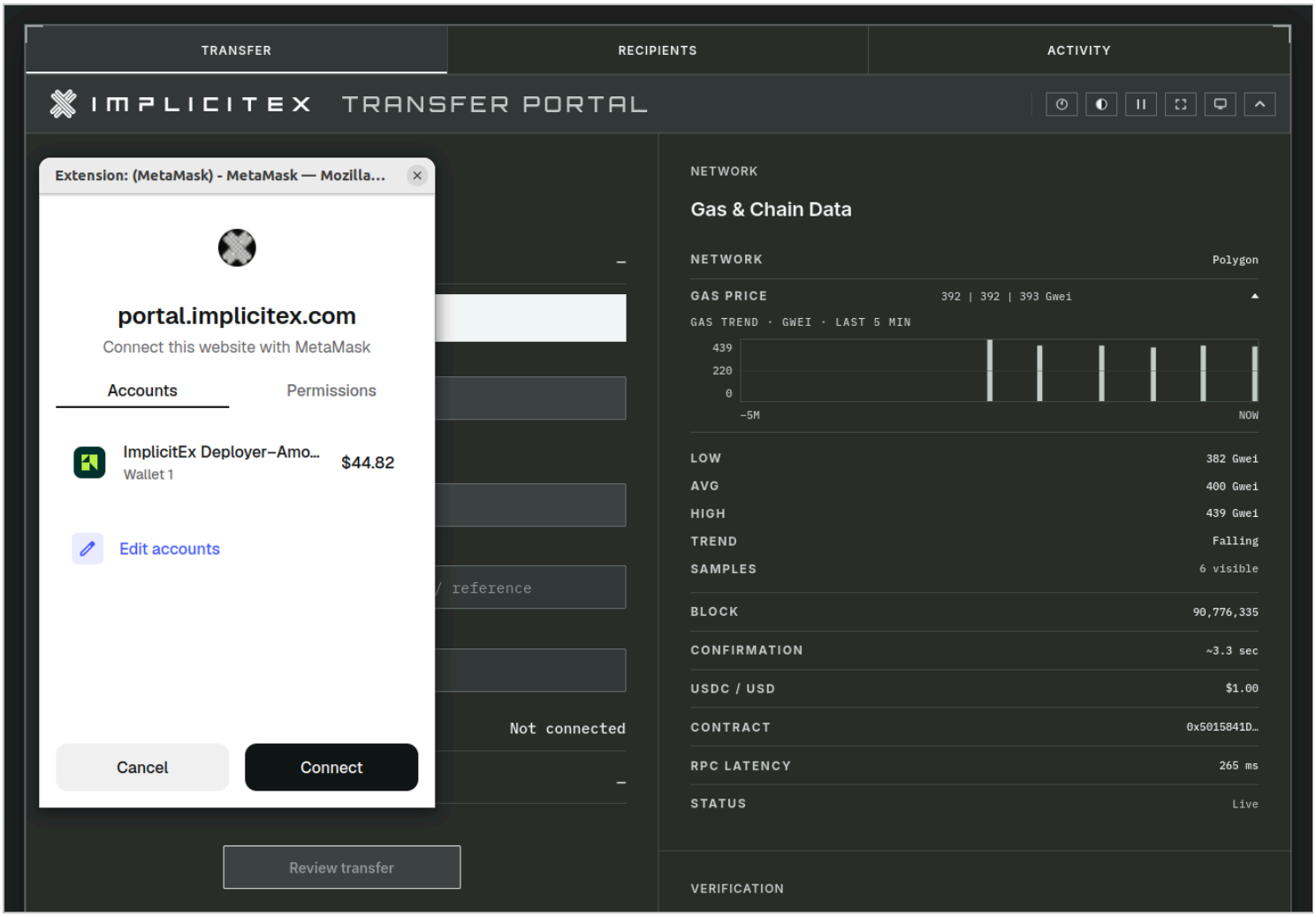

Figure 2

MetaMask displayed an explicit connection request for `portal.implicitex.com`.

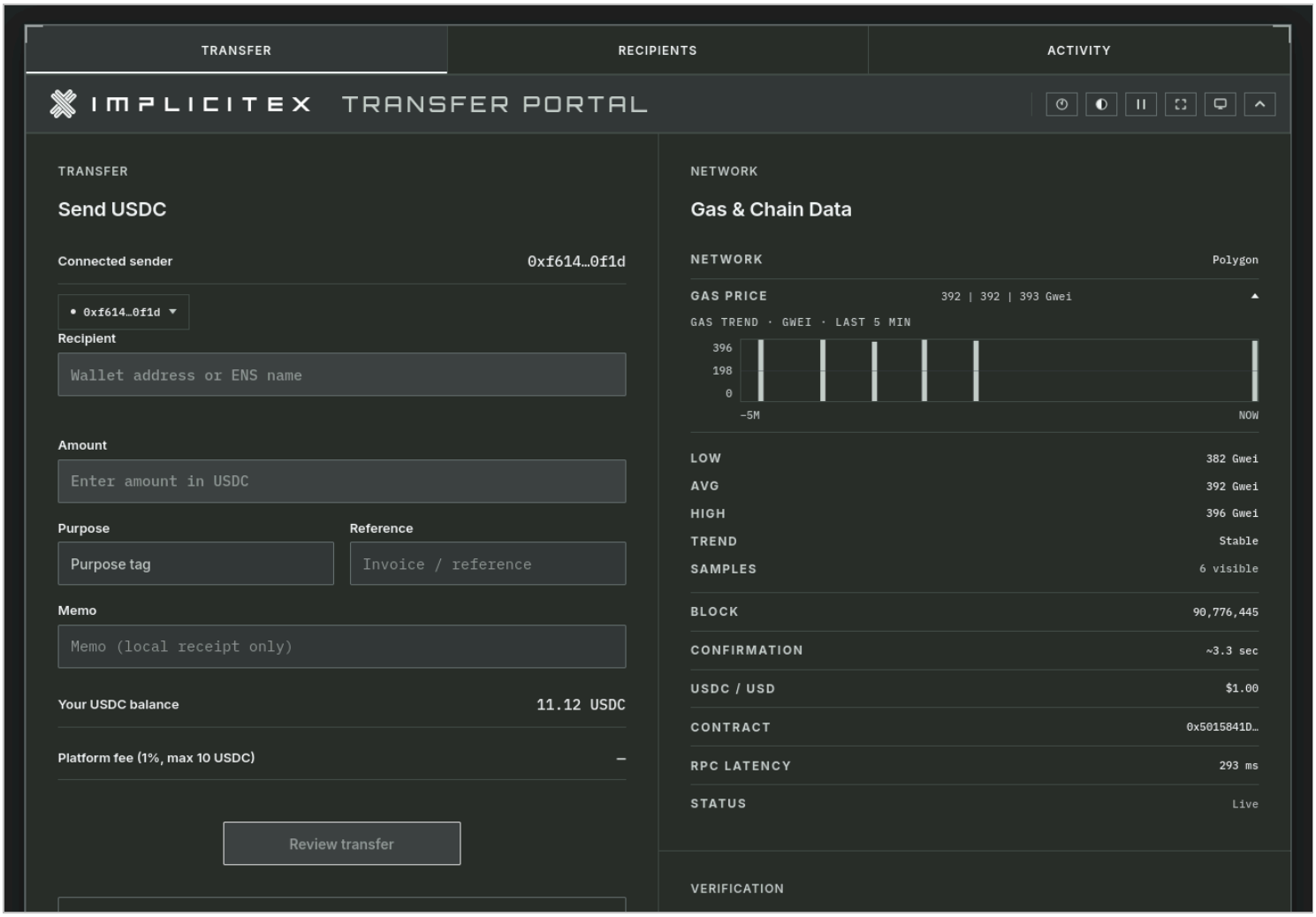

Figure 3

The sender connected. The portal loaded an 11.12 USDC balance and the transfer form became active.

7.2 Recipient validation and fee calculation

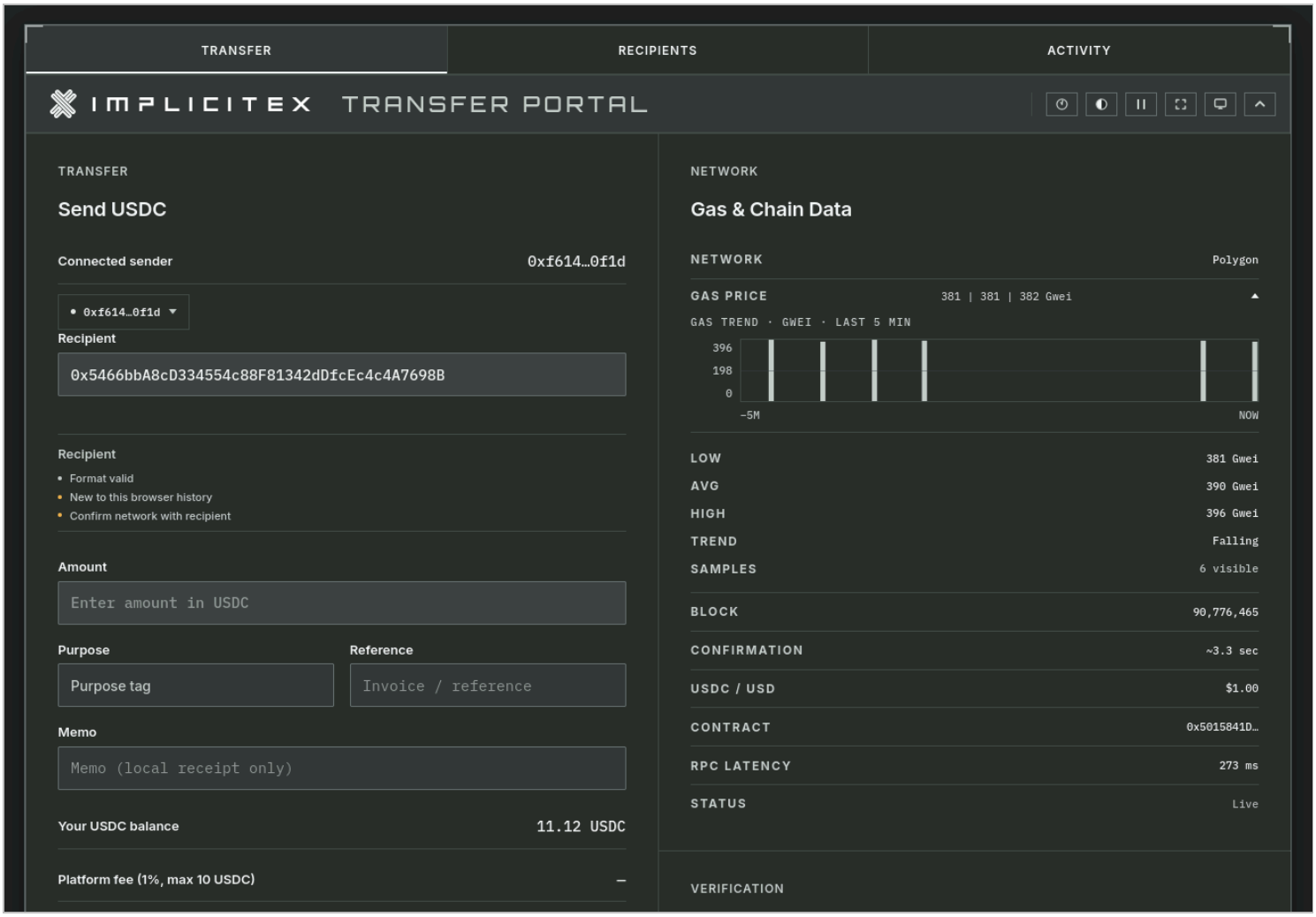

Figure 4

The recipient address was entered and passed format validation.

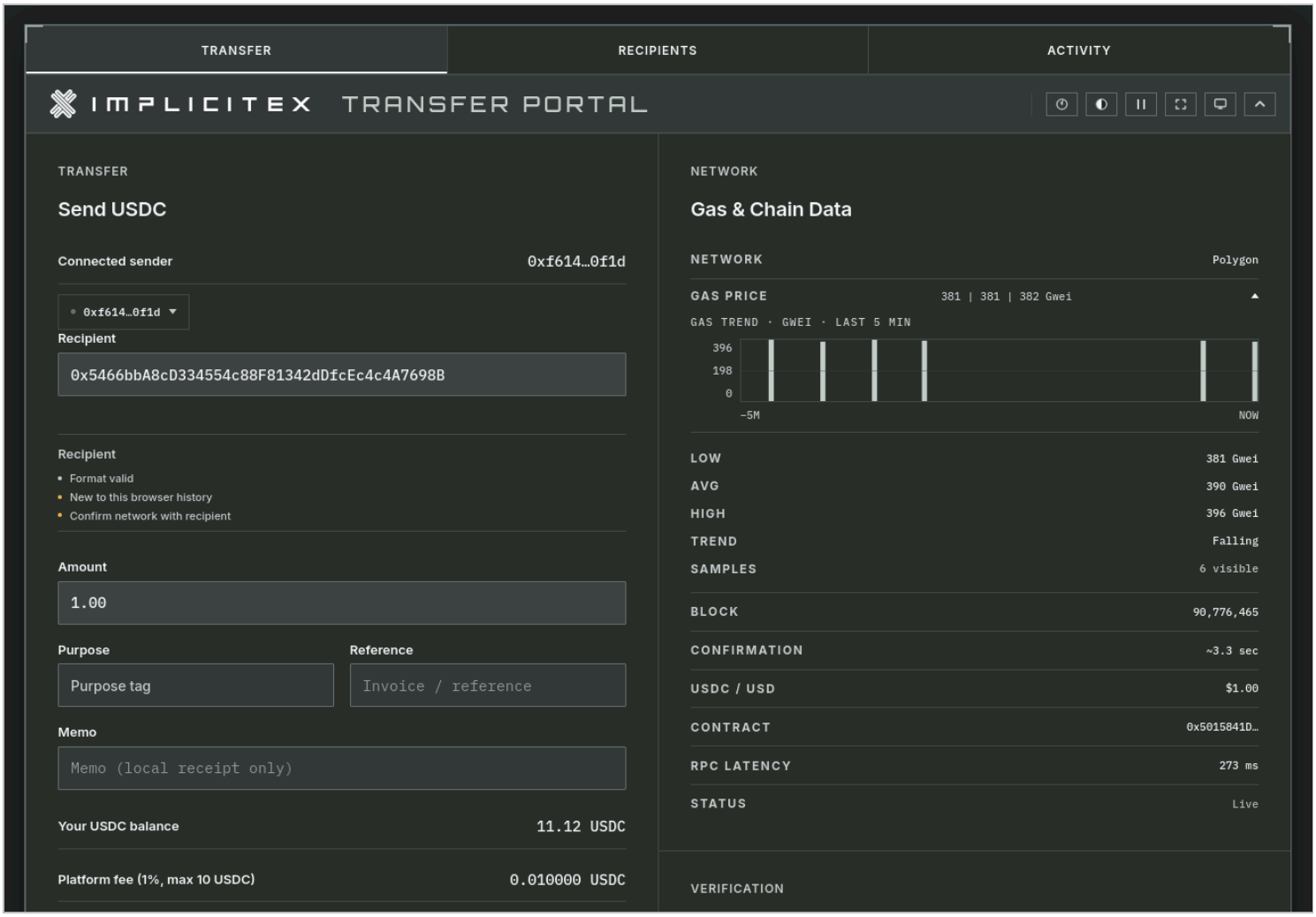

Figure 5

The 1.00 USDC amount produced a 0.010000 USDC platform fee.

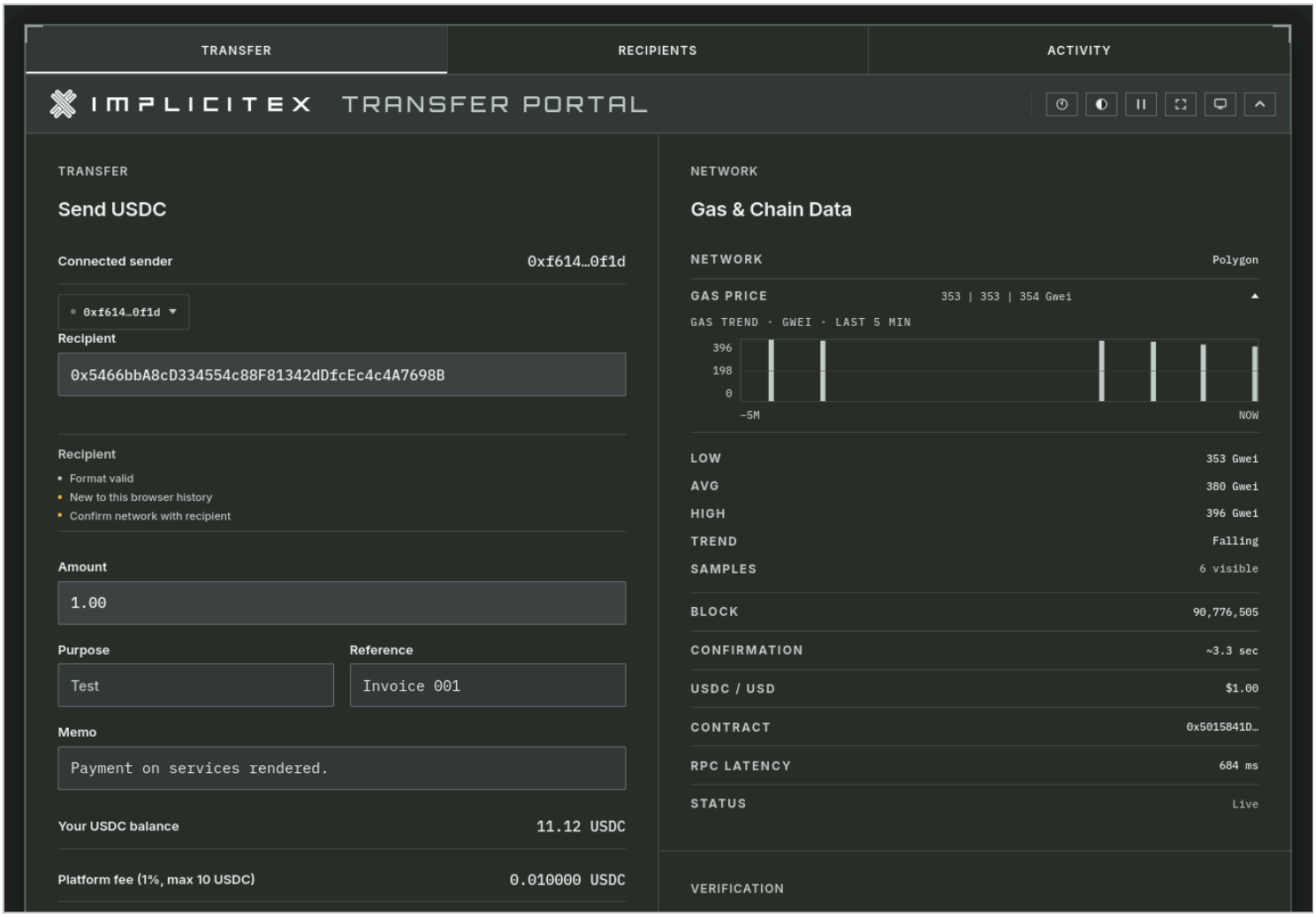

Figure 6

The transaction details were fully populated (recipient, amount, purpose, reference, memo) before execution review.

7.3 Explicit user confirmation

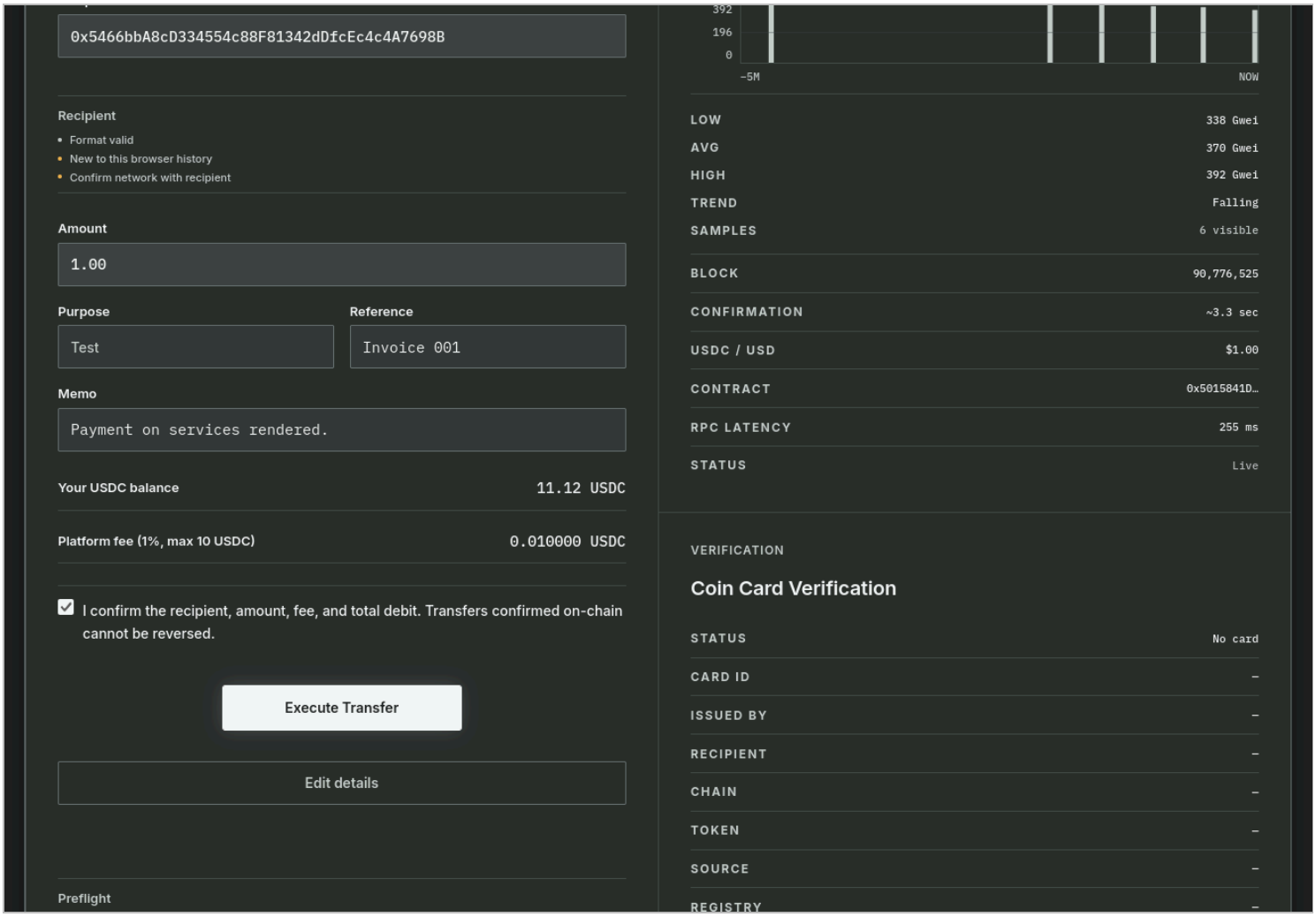

Figure 7

The user explicitly acknowledged the recipient, amount, fee, total debit, and irreversibility. Execute Transfer became available only after the checkbox was checked.

7.4 ERC-20 approval request and observed wallet warning

The portal requested a USDC allowance equal to the current transfer total. The requested cap was not unlimited and did not include an additional buffer.

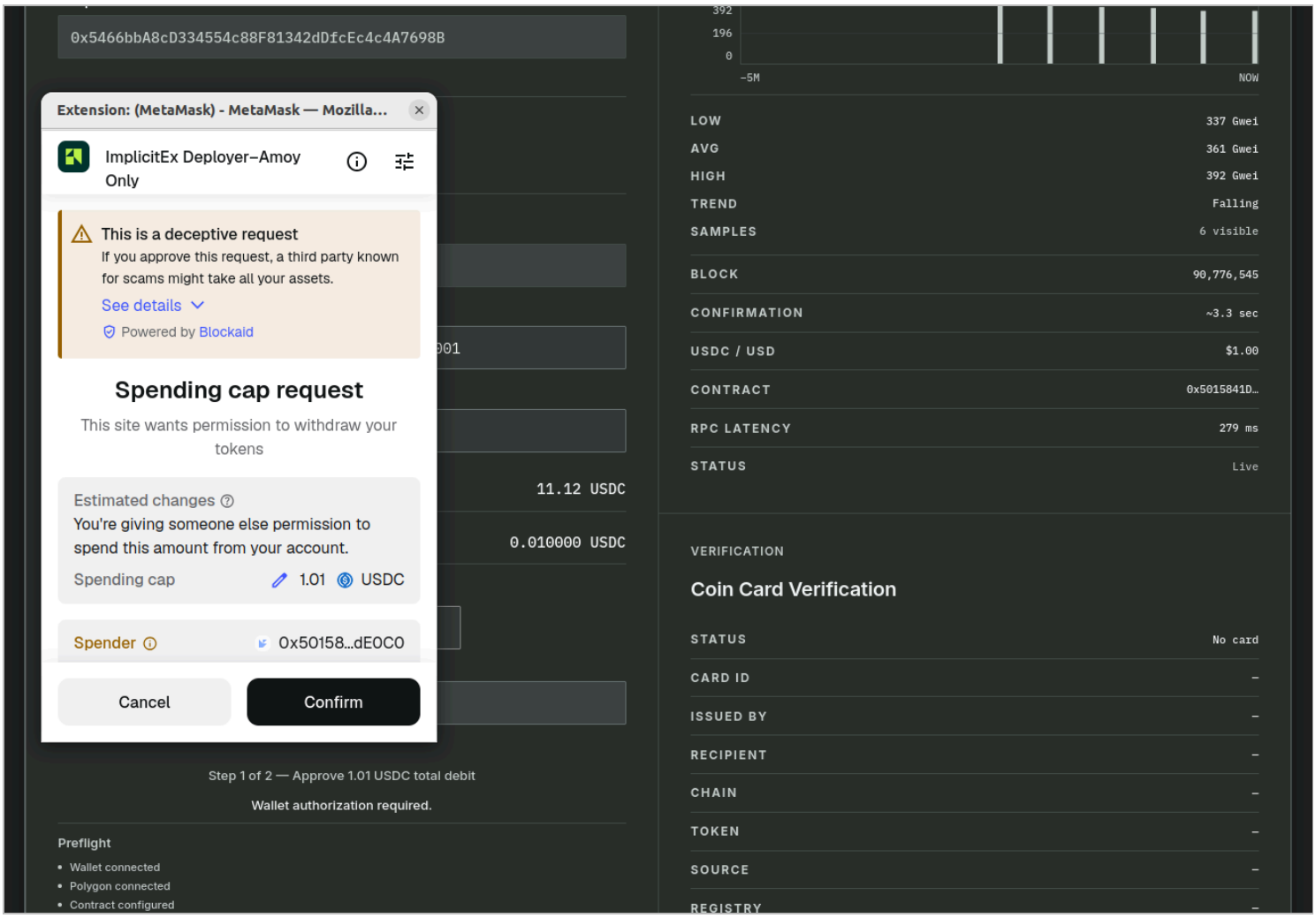

Figure 8

During the ERC-20 approval step, MetaMask displayed a Blockaid warning indicating "This is a deceptive request." The approval requested an exact spending cap of **1.01 USDC**, matching the disclosed transfer amount (1.00 USDC) plus the platform fee (0.01 USDC). The subsequent transfer transaction and on-chain confirmation proceeded normally.

7.5 On-chain confirmation

After approval, the portal issued the separate `transferWithFee()` call. The transfer was confirmed on Polygon at block 90,776,572.

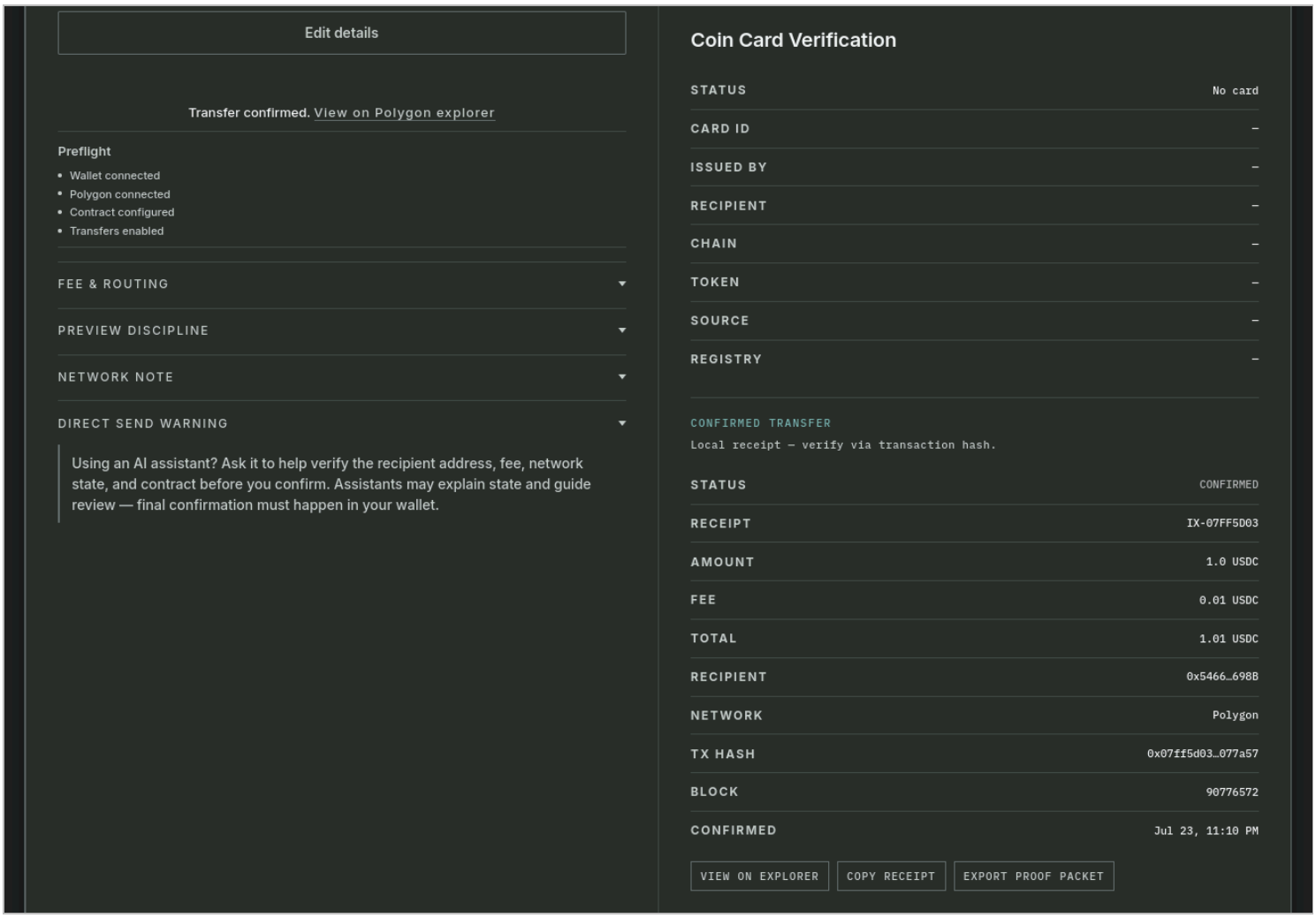

Figure 9

The portal displayed the confirmed transfer: receipt IX-07FF5D03, transaction hash `0x07ff5d03…077a57`, block 90,776,572, confirmed Jul 23, 11:10 PM. Actions available: View on Explorer, Copy Receipt, Export Proof Packet.

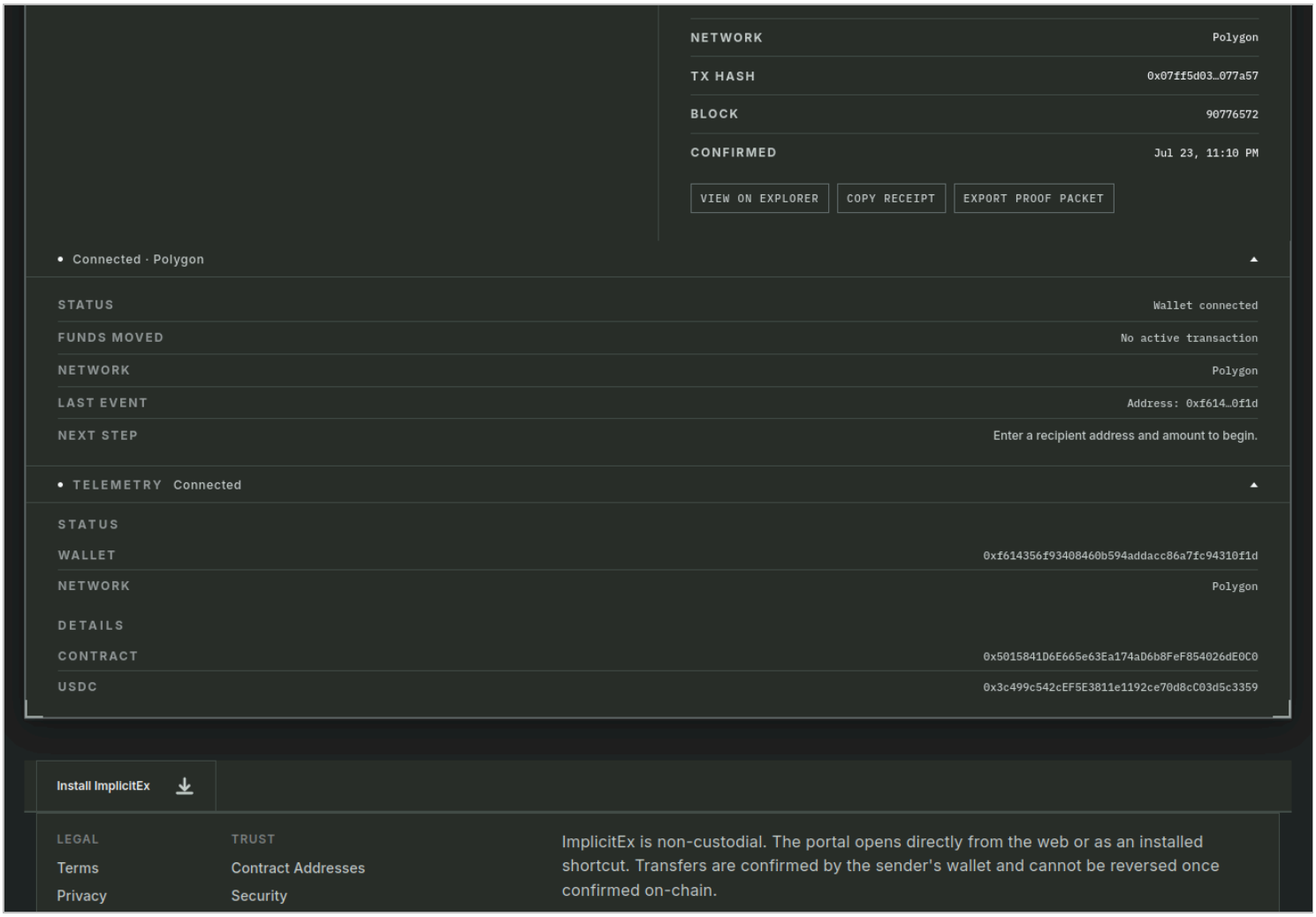

Figure 10

The companion tray confirmed the transaction (TX HASH, BLOCK 90,776,572). The telemetry panel recorded the full contract address, USDC address, and wallet. No active transaction was pending.

7.6 Activity record and on-chain verification

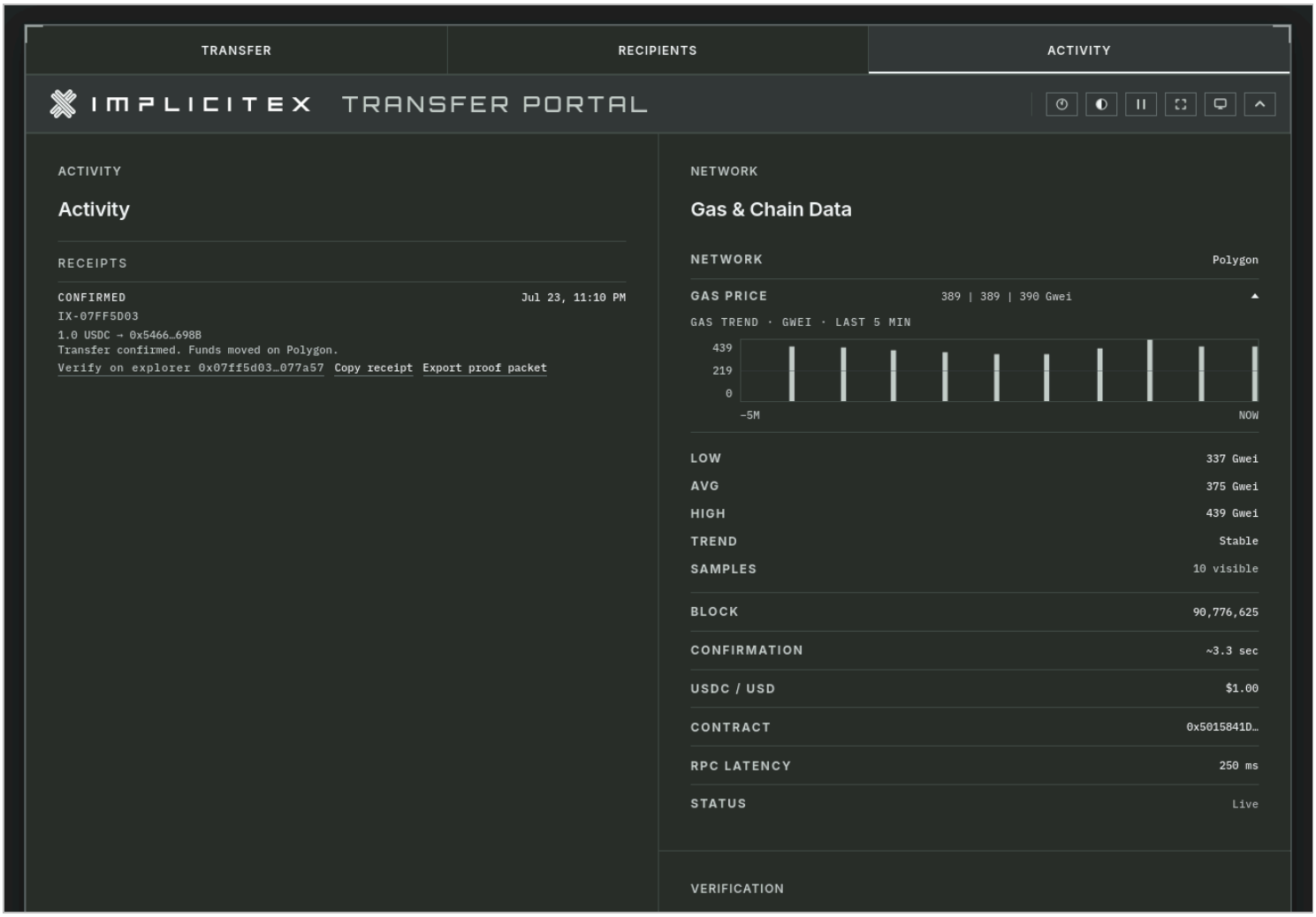

Figure 11

The Activity tab recorded receipt IX-07FF5D03 as **CONFIRMED**. "Transfer confirmed. Funds moved on Polygon." The explorer link is available for independent verification.

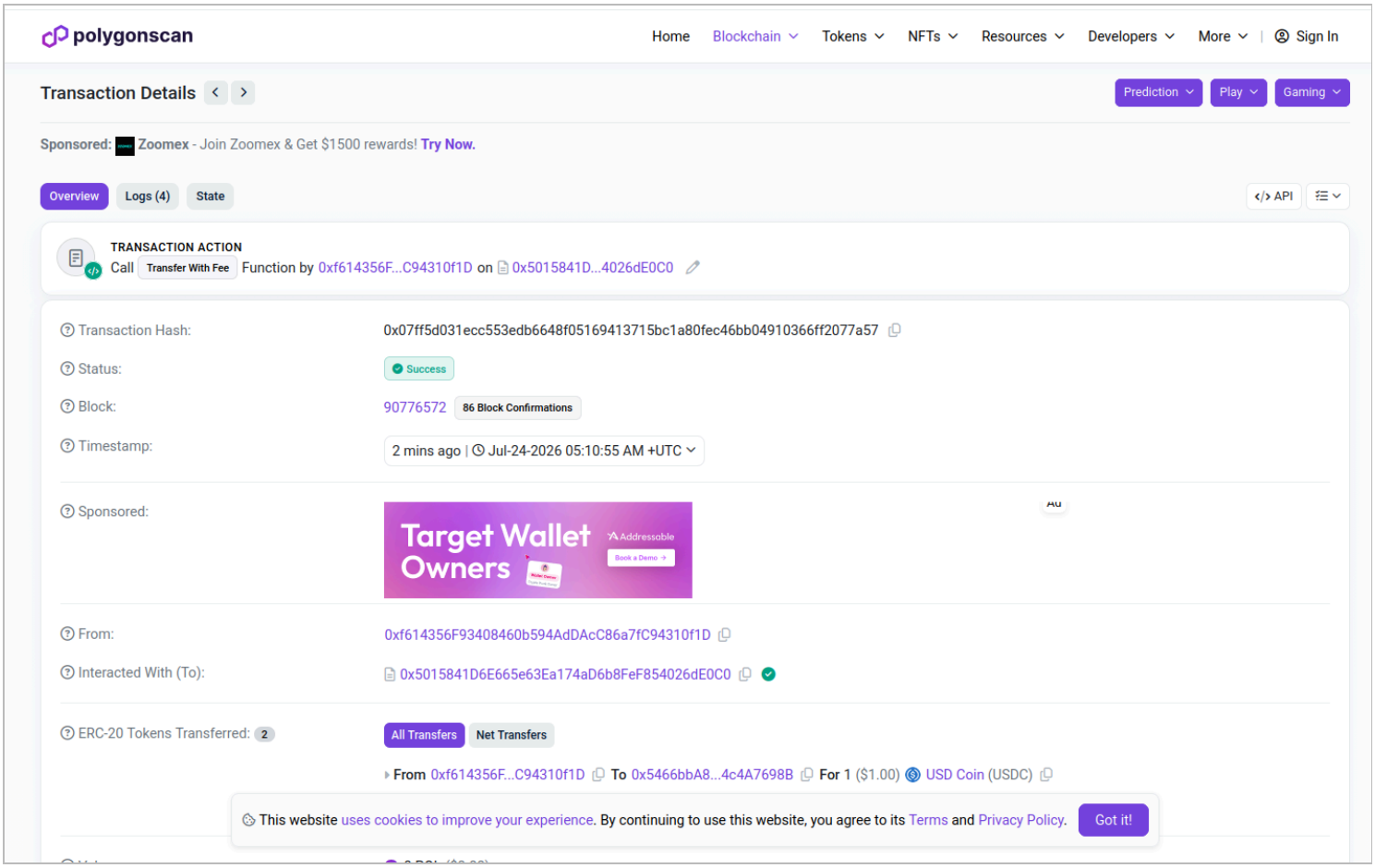

Figure 12

Polygonscan confirmed the `transferWithFee()` call: status **Success**, block 90,776,572, 86 confirmations at capture. The ERC-20 transfer record shows 1.00 USDC from sender `0xf614…f1D` to recipient `0x5466…698B`.

7.7 Production settlement evidence — two execution paths

Two production execution paths were exercised against the deployed system and each is supported by an exact browser-exported proof packet preserved byte-for-byte at time of export. Together they establish that both paths produce a consistent, independently verifiable settlement record.

7.7.1 Original Blockaid-observed approval-path transaction (2026-07-23)

Documents the first-time-user flow visible in Figures 1–12 above. The wallet had no prior allowance; an ERC-20 approval was required. The Blockaid spending-cap warning appeared during the approval step (Figure 8) and did not prevent settlement. This transaction is independently verifiable on Polygonscan. The original browser-exported proof packet for this transaction was not preserved at time of export; the chain record and screenshot evidence stand as the independent verification record.

Transaction hash:	`0x07ff5d031ecc553edb6648f05169413715bc1a80fec46bb04910366ff2077a57`
Approval hash:	`0xb137bebac424d6f39630c00938776f1397450a0c531523432052aa57d001f7fb`
Explorer	Polygonscan transaction record
Status	`confirmed`
Block number	90,776,572

Funds moved	`true`
Evidence	Figures 1–12; Polygonscan; artifact gap noted — original browser JSON not preserved
Execution path	READY → AUTHORIZING → AUTHORIZED → SUBMITTING → SUBMITTED → CONFIRMED

The approval transaction calldata was independently verified on-chain: selector `0x095ea7b3` (`approve`), target USDC contract (`0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359`), spender the ImplicitEx transfer contract (`0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0`), approved value `1010000` (1.010000 USDC). This value matches the portal review total displayed in Figures 5–8: 1.00 USDC transfer plus 0.01 USDC platform fee.

The transfer transaction emitted two ERC-20 Transfer events from the USDC contract: 1.000000 USDC from sender directly to recipient, and 0.010000 USDC from sender directly to treasury. Both transfers originate from the sender's address. The supported execution path does not route the transferred amount through the ImplicitEx contract.

7.7.2 Approval-path certification run (2026-07-24)

A dedicated approval-path certification run was executed to close the artifact gap. The wallet allowance was zeroed to 0 USDC before execution, forcing the full approval lifecycle. The exported proof packet is the browser-generated artifact, preserved byte-for-byte immediately after confirmation. Reference: `APPROVAL-PATH-002`.

Transaction hash:	`0x48c07f8039d50754a20c1b32ee701fa042fcb4bbeabd8d01ab0164805c5faa78`
Approval hash:	`0x115647e9ec9cab18e1bd422a68d5fd4216383fa240b58eb02f53e44274a7f428`
Explorer	Polygonscan transaction record
Status	`confirmed`
Block number	90,784,121
Funds moved	`true`
Purpose / reference	`test` / APPROVAL-PATH-002
Execution path	READY → AUTHORIZING → AUTHORIZED → SUBMITTING → SUBMITTED → CONFIRMED

The exported proof packet for this certification run includes all settlement fields, user-entered metadata, timestamps, and the three evidence sections from the `proof-packet.v1` schema. The approval transaction targets the USDC contract with spender set to the ImplicitEx contract and an approved value of exactly 1.01 USDC — matching the disclosed total debit. On the approval-path branch, the exact 1.010000 USDC allowance is fully consumed by the transfer; post-transfer allowance was independently read back on-chain at 0.000000 USDC. Zero residual allowance is a property of the exact-approval branch: a wallet that enters through the transfer-only branch retains whatever valid allowance remains after the 1.010000 USDC deduction.

7.7.3 Transfer-only path transaction (2026-07-24)

Documents the returning-user flow: wallet had an existing allowance of 3.03 USDC established via a direct contract call. The portal detected sufficient allowance and submitted the transfer directly — no approval step, no MetaMask spending-cap

prompt, no Blockaid warning. The exported proof packet is the browser-generated artifact, preserved byte-for-byte.

Transaction hash:	`0x0e0b3c1f2d0565663743050f43282e97501b3ee8505084fa5af823da00b1e15a`
Approval hash:	`null` — no approval occurred
Explorer	Polygonscan transaction record
Status	`confirmed`
Block number	90,780,021
Funds moved	`true`
Purpose tag	`test`
Reference	TRANSFER-ONLY-001
Memo	Production allowance verification
Execution path	READY → SUBMITTING → SUBMITTED → CONFIRMED

The exported proof packet for this transaction includes all settlement fields, user-entered metadata, timestamps, and the three evidence sections from the `proof-packet.v1` schema.

The preexisting 3.030000 USDC allowance decreased to 2.020000 USDC after the transfer — exactly matching the 1.010000 USDC total debit — confirming that the correct amount was consumed and no phantom spending occurred. The remaining 2.020000 USDC is the unused portion of the preexisting 3.030000 USDC allowance. The transfer neither created nor increased the allowance; it consumed exactly the disclosed 1.010000 USDC total debit.

8 INTERNAL VERIFICATION EVIDENCE

SCOPE OF THIS SECTION

The evidence below reflects internal testing executed against the deployed application. It has not been reviewed or certified by an independent third party. The test suite results are developer-run. The Gate 4 transaction is independently verifiable on Polygonscan.

8.1 Test suite results

Suite	Tests passed	Result
Contract (Hardhat / Chai)	66 / 66	PASS
Receipt lifecycle + metadata	14 / 14	PASS
Observability	31 / 31	PASS
Analytics	24 / 24	PASS

Suite	Tests passed	Result
Consent	15 / 15	PASS
Portal integrity	—	PASS
Static reference audit	631 references checked	PASS

8.2 Contract test coverage — areas exercised

The contract test suite (66 tests, Hardhat / Chai) covers the following scenarios:

- Constructor validation — zero-address rejection for USDC and treasury; fee cap enforcement on initial fee basis points
- Treasury management — address update, event emission, zero-address guard
- Fee configuration — floor boundary, cap boundary (MAX_FEE_BPS), event emission
- Transfer execution — happy path, fee routing correctness, direct balance checks
- Pause and unpause — state enforcement (transfers blocked when paused), event emission
- Ownership transfer — two-step requirement (acceptance required)
- Recipient validation — zero-address rejection, contract-address rejection
- Reentrancy protection — reentrant call behavior under `nonReentrant`
- `previewTransfer()` accuracy — fee and total-debit calculation
- `rescueERC20` — USDC rejection, non-USDC token recovery
- Allowance and balance edge cases — insufficient balance, insufficient allowance

8.3 Gate 4 — Mainnet controlled live smoke (2026-06-15)

A controlled transfer was executed on Polygon mainnet from the production frontend. This is the primary on-chain evidence of correct fee routing and zero-custody behavior.

```
Date:           2026-06-15
Network:        Polygon mainnet, chain ID 137
Site:           https://implicitex-236f2.web.app
Contract:       0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0

Sender:         0x2489587C9da6EaB970a5479BA70273BA37961221
Recipient:      0xe0B02A6d9738aa36eE48004211E264b7a815796B
Treasury:       0xa7cE4232811021d2Dd01f4f0f264Df2427ab3919

Amount:         1.000000 USDC
Fee:            0.010000 USDC  (100 basis points)
Total debit:    1.010000 USDC

Transaction:    0x37fd733a7f1854740bf702aa5bf59794f4ebab0f39d2a84fb2c231c29df622d9
Block:          88565097

Polygonscan:    https://polygonscan.com/tx/0x37fd733a7f1854740bf702aa5bf59794f4ebab0f39d2a84fb2c231c29df622
```

Flow	Expected	Result	Source
Sender deducted	1.010000 USDC	Confirmed	Wallet balance delta
Recipient received	1.000000 USDC	Confirmed	Polygonscan ERC-20 log
Treasury received	0.010000 USDC	Confirmed	Polygonscan ERC-20 log
Contract retained	0.000000 USDC	Confirmed	Polygonscan contract balance

The Polygonscan ERC-20 token transfer log for this transaction shows exactly two transfers originating from the same sender address: 1.000000 USDC to the recipient and 0.010000 USDC to the treasury. No additional transfers are present.

8.4 Gate 5 — Public launch attempt (2026-06-18)

Transfers were enabled for a first-user walkthrough on 2026-06-18. The walkthrough was halted at the USDC approval step (prompt 1 of 2) due to the Blockaid "deceptive request" classification. The transfer execution step (prompt 2 of 2) was not reached. No funds moved. Transfers were returned to disabled standby state on the same date.

The false-positive report was submitted to Blockaid on 2026-06-18 for both `implicitex.com` and `implicitex-236f2.web.app`.

9 KNOWN LIMITATIONS

The following limitations apply to the currently deployed version. They are stated here for completeness and accuracy.

Item	Current status
Third-party security audit	Not yet completed. An independent audit is planned as the platform scales toward broader distribution. Internal test coverage is documented in Section 8 and is not a substitute for an external audit.
Transfer reversibility	Confirmed blockchain transactions cannot be reversed. ImplicitEx has no reversal, refund, or cancellation mechanism for executed transfers.
Network support	Polygon mainnet (chain ID 137) only. Other networks are not supported in the current deployment.
Transfer ceiling	250.00 USDC per transaction. This is a soft launch cap enforced in the portal; it is not a contract-level constraint.
Mobile wallet compatibility	MetaMask browser extension (desktop) is confirmed. MetaMask Mobile in-app browser is under active testing. A provider-sequencing issue affecting the transfer execution prompt (prompt 2 of 2) has been diagnosed and a fix has been deployed; device verification is pending.
WalletConnect	Integration is implemented and active in the production codebase. Full regression testing against WalletConnect wallet providers is ongoing.

Item	Current status
Per-transfer fee ceiling	The deployed contract does not enforce an absolute fee ceiling in USDC terms. The fee formula is a flat 1% percentage with no upper bound enforced in contract code. ImplicitEx has adopted a future policy under which fees will be capped at 10.00 USDC for transfers above 1,000 USDC, but that policy has not been deployed in the current contract. The production portal's 250.00 USDC transfer ceiling prevents any user of the supported interface from reaching the amount at which the uncapped 1% formula would exceed 2.50 USDC. A future contract revision will implement the 10.00 USDC ceiling on-chain before the portal transfer ceiling is raised above 1,000 USDC.
Wallet-provider classification	During the observed 2026-07-23 flow, MetaMask displayed a Blockaid warning indicating "This is a deceptive request" at the USDC spending-approval step. The approval requested an exact 1.01 USDC spending cap, matching the disclosed transfer amount (1.00 USDC) plus the platform fee (0.01 USDC). The subsequent transfer transaction was confirmed on-chain at block 90,776,572 with funds moved. See §7.4 and §7.7.1 for the full evidence record.
Contract ownership	Single-key `Ownable2Step`. Multi-signature governance is on the planning roadmap but is not deployed.

10 REVIEWER REPRODUCTION PROCEDURE

The following verification steps can be completed independently by a Blockaid analyst without wallet connection or access to the ImplicitEx codebase.

10.1 Contract source

The deployed contract is at `0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0` on Polygon mainnet (chain ID 137).

The source is verified on Polygonscan. The matched Solidity source, compiler version, optimizer settings, ABI, and constructor arguments are publicly visible at:

```
https://polygonscan.com/address/0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0#code
```

The contract's verified-source status was confirmed during the deployment procedure on 2026-05-22. A Hardhat `verify` invocation returned that the contract had already been verified on the block explorer. Polygonscan reports that the published source compiles to bytecode matching the deployed contract, and publicly exposes the matched Solidity source, compiler configuration, ABI, and constructor arguments for the deployed address. A reviewer can inspect these directly on Polygonscan without any supplemental file attachment.

10.2 Transaction verification

The Gate 4 controlled smoke transaction is publicly accessible:

```
https://polygonscan.com/tx/0x37fd733a7f1854740bf702aa5bf59794f4ebab0f39d2a84fb2c231c29df622d9
```

In the "ERC-20 Token Txns" tab on Polygonscan, two transfers are visible originating from the same sender:

- To `0xe0B02A6d...796B` — 1.000000 USDC (recipient)
- To `0xa7cE4232...3919` — 0.010000 USDC (treasury fee)

10.3 Contract state — read-only queries

The following state can be read from the deployed contract via Polygonscan "Read Contract" or any JSON-RPC client, without connecting a wallet:

Function	Expected return
`feeBasisPoints()`	100 (1.00%)
`MAX_FEE_BPS()`	100 (hard-coded fee rate ceiling; owner cannot set feeBasisPoints above this value)
`minTransferAmount()`	1000000 (1.000000 USDC)
`treasury()`	`0xa7cE4232811021d2Dd01f4f0f264Df2427ab3919`
`paused()`	Current pause state
`owner()`	`0x776A0D6b9F96445A38303F56d5B923e6d1FF8E97`

10.4 Live approval flow — reviewer reproduction

Transfers are currently enabled on Polygon mainnet. The approval flow can be reproduced as follows:

1. Connect a Polygon wallet with at least 2 USDC to `https://portal.implicitex.com`
2. Enter any valid EOA recipient and an amount (e.g. 1.00 USDC)
3. Proceed past the review screen
4. Observe the MetaMask spending-cap prompt

When the approval path is triggered, MetaMask's spending-cap value will equal the portal's displayed total debit exactly: transfer amount plus the disclosed 1% platform fee, with no additional allowance buffer. For a 1.00 USDC transfer, the spending cap will be 1.01 USDC. The precise classifier basis for Blockaid's warning has not been provided by Blockaid; this packet documents observable behavior and reproducible transaction evidence rather than attributing an unconfirmed cause.

11 ADDRESSES AND EVIDENCE INVENTORY

11.1 Deployed addresses — Polygon mainnet (chain ID 137)

Component	Address
ImplicitExTransfer contract	`0x5015841D6E665e63Ea174aD6b8FeF854026dE0C0`
USDC (Circle native, Polygon)	`0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359`

Component	Address
Treasury (fee recipient)	`0xa7cE4232811021d2Dd01f4f0f264Df2427ab3919`
Contract owner	`0x776A0D6b9F96445A38303F56d5B923e6d1FF8E97`

11.2 Deployment record

Parameter	Value
Deployment date	2026-05-22
Deployment transaction:	`0x87593fdb3d256a4a94b3e73877ba0bc433c39e81eefc78334af6da79ff5ef1f3`
Ownership transfer TX:	`0xd84181dbffbd4f760cfa650b2a1edb1acc17713e66bb5d6d478376dc5202d777`
Ownership acceptance TX:	`0xd6bfb2876725391c956dbd17ec5f774f9246b50df5667e8b29e8c78305365e90`

11.3 Evidence inventory

ID	Item	Status
TF-01	Ordered transaction-flow screenshot set (Figures 1–12) — approval path	Available — confirmed flow captured 2026-07-23
PP-01	`proof-packet.v1` export — transfer-only path (exact browser export) Artifact: transaction-proof-packet-transfer-only-2026-07-24.json TX: `0x0e0b3c1f2d0565663743050f43282e97501b3ee8505084fa5af823da00b1e15a` · Block: 90,780,021 SHA-256: `541cbb3d41788648c0f8907475ed542dcd3f7899fd736c9ad21d44bbc3bd6da7`	Publicly served — exact browser export, 2026-07-24; `approvalHash: null`; metadata preserved
PP-02	`proof-packet.v1` export — approval-path certification run (exact browser export) Artifact: transaction-proof-packet-approval-2026-07-24.json TX: `0x48c07f8039d50754a20c1b32ee701fa042fcb4bbeabd8d01ab0164805c5faa78` · Block: 90,784,121 Approval hash: see §7.7.2 table SHA-256: `ae91d64b50d20b28df8c9bcd8bd2ecfac595f872b7c65cfbd1bb98bd2ffdbf7a`	Publicly served — exact browser export, 2026-07-24; `referenceId: APPROVAL-PATH-002`; see §7.7.2
TX-02	Original Blockaid-observed approval-path transaction record — 2026-07-23	Confirmed — block 90,776,572; Polygonscan; Figures 1–12; artifact gap noted (original browser JSON not preserved); see §7.7.1

ID	Item	Status
TX-01	Gate 4 transfer transaction (Polygonscan)	Available — publicly verifiable
TC-01	Contract test suite results (66/66)	Internal — available on request
TC-02	Observability suite results (31/31)	Internal — available on request
TC-03	Analytics suite results (24/24)	Internal — available on request
TC-04	Static reference audit (631 references)	Internal — available on request

11.4 Pre-submission checklist

☐ Contract source verified on Polygonscan — confirmed 2026-05-22. Polygonscan source link included in Section 10.1. No exhibit attachment required.

☐ Record current contract state snapshot via Polygonscan "Read Contract" and verify each value matches the corresponding claim in Sections 5 and 11. The contract is not deployed behind an upgradeable proxy, and its runtime code is not administratively upgradeable; configurable state values may nevertheless change.

- ☐ `feeBasisPoints()` — expected: 100 (1.00%)
- ☐ `MAX_FEE_BPS()` — expected: 100 (compile-time constant; fee rate ceiling)
- ☐ `treasury()` — expected: 0xa7cE4232811021d2Dd01f4f0f264Df2427ab3919
- ☐ `owner()` — expected: 0x776A0D6b9F96445A38303F56d5B923e6d1FF8E97
- ☐ `paused()` — record current value; note in packet if state differs from deployment baseline
- ☐ `minTransferAmount()` — expected: 1000000 (1.000000 USDC)

☐ Verify the current portal operating policy against the deployed configuration (`chains.js`), including the 250.00 USDC transfer ceiling and the global and Polygon-specific `transfersEnabled` values. These are frontend operating controls, not on-chain contract constraints. Record the production revision or commit used for the comparison.

☐ Confirm Gate 4 transaction (TX-01) resolves on Polygonscan

☐ Review the final rendered document in a browser: check page breaks, address wrapping, screenshot legibility, and print output on a monochrome printer

11.5 Post-release follow-up (not a submission blocker)

- **Receipt-display parity gap:** The portal's human-readable Activity receipt for the PP-02 certification run (IX-48C07F80) displays `purposeTag` and `referenceId` but omits the `memo` field and the `approvalHash`, even though both are present and correct in the exported JSON proof packet. The JSON is authoritative; the display omission does not affect artifact fidelity. Fix: surface `memo` and `approvalHash` in the receipt UI to eliminate the parity gap between what the portal shows and what the proof packet records.